PYTHON WEB DEVELOPMENT • MODULE 1

Introduction to Flask & Web Development Concepts

Presented by Prof. Nilesh Panchotiya

Learning Objectives

- Master fundamental web architecture concepts
- Differentiate Frontend vs Backend operations
- Understand Client-Server and 3-Tier models
- Learn HTTP Request / Response transactions
- Understand why web frameworks are used
- Compare Flask (Micro) vs Django (Full-Stack)
- Explore Flask Request Flow & MVC structure
- Build and run your first multi-route Flask application

Web Development Fundamentals

Static Websites

Delivers fixed HTML/CSS content stored on server directly to browser without server execution.

- Fast & simple to build
- Examples: Portfolios, static landing pages

Dynamic Web Applications

Generates custom content in real-time using backend code (Python/Flask) and database records.

- Personalised per user
- Examples: Amazon, YouTube, Instagram, Gmail

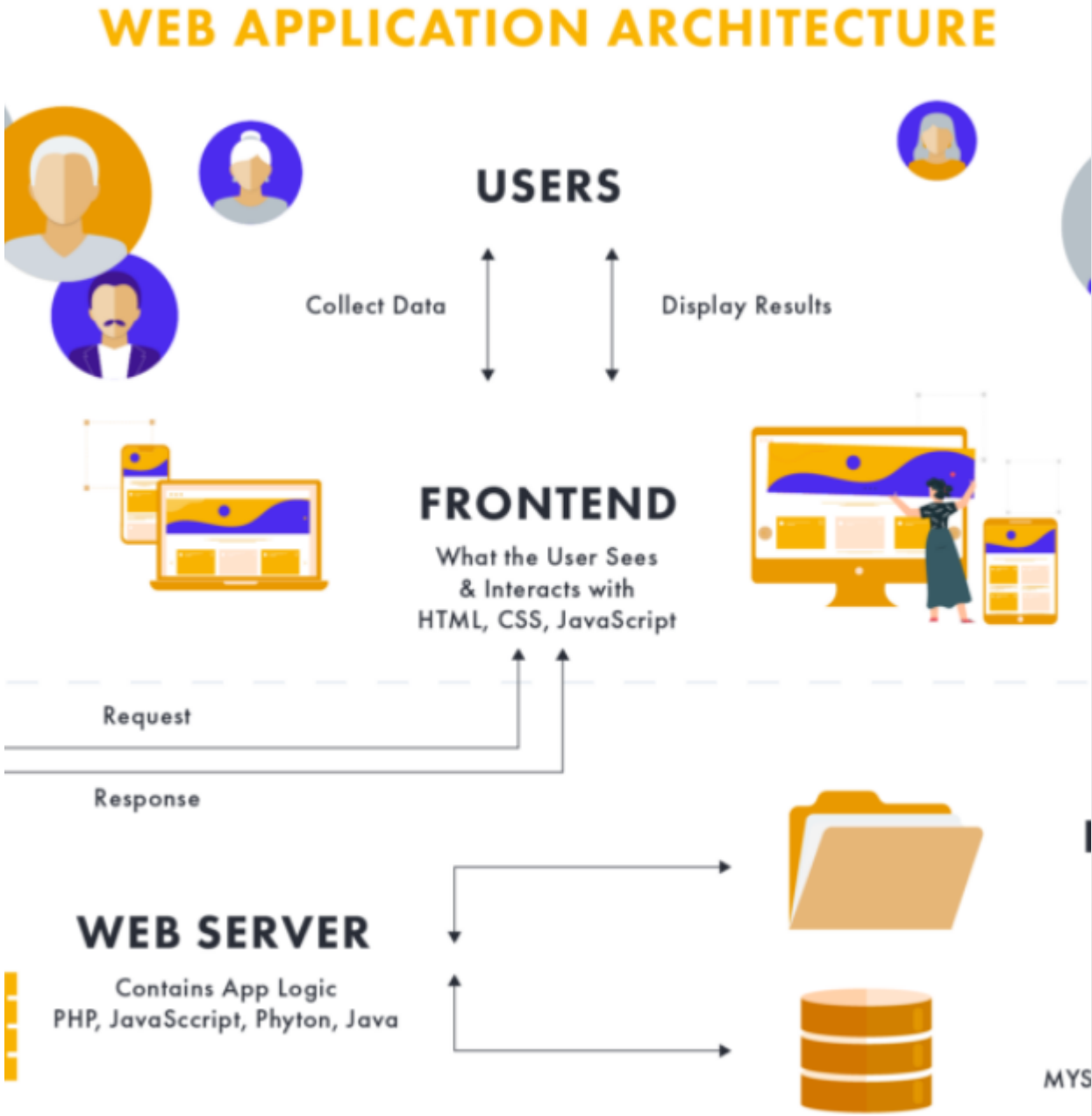
Frontend vs Backend

Frontend (Client-Side)

The visual interface user interacts with. Built using HTML, CSS, JavaScript, React, Angular.

Backend (Server-Side)

The hidden engine executing business logic, security, and database queries. Powered by Python & Flask.



Client-Server Architecture

1. Client (Browser)

Captures user action, formats HTTP Request, and renders received HTML payload.

2. Server (Flask)

Processes incoming route requests, executes Python code logic, builds response.

3. Data Layer

Stores permanent records (SQLite, MySQL) queried dynamically by the server.

How Web Applications Work



HTTP Protocol & Request Methods

HTTP Method	Core Action	Real-World Example Context
GET	Retrieve / read data from server	Opening Amazon homepage or searching Google
POST	Send new data to server	Submitting a login form or posting a comment
PUT	Update existing resource completely	Updating profile info or editing blog post
DELETE	Remove resource from server	Deleting an email or deleting an uploaded photo

URL Structure & Status Codes

Anatomy of a URL

```
https://amazon.in/mobile
```

Protocol: https:// (Secure encryption)

Domain: amazon.in (Server domain/IP)

Path: /mobile (Flask endpoint route)

Status	Meaning	Context
200	OK	Success
404	Not Found	Invalid URL path
403	Forbidden	No authorization
500	Server Error	Python program crashed

Cookies and Sessions

Cookies (Client Storage)

Small key-value data stored directly inside the user's Browser.

- Keeps track of lightweight user preferences
- Sent with every HTTP request automatically

Sessions (Server Storage)

Encrypted data state stored securely on Backend Flask Server.

- Remembers logged-in users across page reloads
- Prevents re-entering password repeatedly

Introduction to Web Frameworks

Building Without a Framework

Developers must manually write raw socket handling, HTTP message parsing, manual URL matching, and header management from scratch.

Building With Flask Framework

Framework provides pre-built routing, session handling, templating, and request processing out-of-the-box so you focus purely on app logic.

Flask Features & Background



Micro Framework

Lightweight core created by Armin Ronacher in 2010. Simple, flexible, unopinionated.



Built on Werkzeug & Jinja2

Uses Werkzeug for WSGI routing/HTTP handling & Jinja2 for dynamic HTML templating.



Built-in Server & Debugger

Includes development web server and interactive debugger for rapid local testing.

Flask vs Django Overview

Comparison Feature	Flask (Micro Framework)	Django (Full-Stack Framework)
Philosophy	Minimalist & Unopinionated	"Batteries Included" (Monolithic)
Setup Complexity	1 File (`app.py`), 5 lines code	Multiple pre-generated config files
Database Layer	Pluggable (SQLAlchemy, Mongo)	Built-in Django ORM
Best Suited For	APIs, ML Deployment, Microservices	Large Enterprise monoliths

When to Choose Flask or Django

Choose Flask When:

- Learning backend concepts without unnecessary complexity
- Building lightweight REST APIs & Microservices
- Deploying Machine Learning models (AI)
- Creating student projects and prototypes

Choose Django When:

- Building large E-Commerce / ERP systems
- Built-in admin panel and user auth are required
- Working with large teams following rigid structure
- Enterprise applications with built-in ORM

Flask Request Processing Flow



Project Structure & MVC

```
my_flask_app/
├── app.py              # Main Python file
├── templates/          # Dynamic HTML files
│   ├── index.html
│   └── about.html
└── static/             # Static assets
    ├── css/style.css
    └── js/script.js
```

Model (Data): Database layer handling data operations (e.g. via SQLAlchemy).

View (Presentation): HTML templates in `templates/` directory rendered to user.

Controller (Logic): Python functions in `app.py` processing user requests.

First Flask Application

```
from flask import Flask # Initialize Flask application app = Flask(__name__) # Define root URL route @app.route("/") def home(): return "Hello Flask!" if __name__ == "__main__": app.run(debug=True)
```

```
# Initialize Flask application
__name__

# Define root URL route
@app.route "/"
def home
return "Hello Flask!"

if "__main__"
True
```

```
y

o.contrib.auth import get_user_model
o.contrib.auth.models import Group

framework import generics, permissions, filters
framework.response import Response
2_provider.contrib.rest_framework import TokenHasReadWriteScope, TokenHa

erializers.auth_serializers import (
stSerializer,
istSerializer,
gnUpSerializer,


our views here.

_user_model()


pList(generics.ListAPIView):
urn a list of all the existing groups.

Authorization with `group` scope required.

sion_classes = [permissions.IsAuthenticated, TokenHasScope]
ed_scopes = ['groups']
et = Group.objects.all()
izer_class = GroupListSerializer


List(generics.ListAPIView):
urn a list of all the existing users.
```

Routing in Flask Applications

@app.route("/") def home(): return "Welcome to Home Page!" @app.route("/about") def about(): return "This is the About Page." @app.route("/contact") def contact(): return "Contact us at: student@college.edu"

```
@app.route    "/"
def   hom
      eturn   "Welcome to Home Page!"

@app.route    "/about"
def   about
      return  "This is the About Page."

@app.route    "/contact"
def   contact
      return  "Contact us at: student@college.edu"
```

Real-World Use & Lab Activity

Companies Using Flask

Netflix: Powers core API microservices

Reddit: Uses Flask for backend traffic services

Lyft: Handles high-concurrency microservices

Hands-on Lab Activity

1. Install Flask: pip install flask
2. Create app.py and add Home, About, Contact routes
3. Run server using python app.py
4. Add a custom /profile route independently

Thank You

Presented by Prof. Nilesh Panchotiya